Function函数

函数是由事件驱动的或者当它被调用时执行的可重复使用的代码块。

当调用该函数时，会执行函数内的代码。

函数的语法

可以在某事件发生时直接调用函数（比如当用户点击按钮时），并且

可由 JavaScript 在任何位置进行调用。

JavaScript 对大小写敏感。关键词 function 必须是小写的，并且必须以

与函数名称相同的大小写来调用函数。

```
//有名函数
    function fn(){
        console.log(265)
        console.log(265)
        console.log(265)
    }
    fn()//第一次调用 =>自执行
    fn()//第二次调用
   //匿名函数    匿名函数；通过var一个变量来定义一个匿名函数；
    var fn123 = function(){
        console.log(2021)
    }
    fn123()//第一次调用 =>他执行:被其它接收的变量来促发的函数，比如事件函数
    fn123()//第二次调用


    function FN1(){}//函数声明
    var FN2 =function(){}//函数表达式

   /* 函数表达式与函数声明的区别
   执行顺序的问题
   函数声明可以先执行后定义
   函数表达式必须先声明再调用[会报错]
   */
     //定义阶段/声明阶段；
     //执行阶段/编译阶段；
```

实参 形参

形参：函数定义/声明，原括号里面的内容

实参：函数自执行时，括号里面的内容；

如果实参个数大于形参个数的话，前面的形参会一一接收到，大于形参的长度的实参不会被接收；

```
//实参：函数自执行时，括号里面的内容；
// fn(666,999,333,"cainiao",'suiyue')
// fn("cainiao",'suiyue','shamo',false)
```

不定参

类数组：所谓的类数组对象：
就是拥有一个 length 属性和若干索引属性的对象。

举个例子：

```
var array = ['house', 'car', 'money'];

var arrayLike = {
    0: 'house',
    1: 'car',
    2: 'money',
    length: 3
}

//读取
console.log(array[0]); // house
console.log(arrayLike[0]); // house

array[0] = 'new house';
arrayLike[0] = 'new house';

//长度
console.log(array.length); // 3
console.log(arrayLike.length); // 3

//遍历
for(var i = 0, len = array.length; i < len; i++) {
   ……
}
for(var i = 0, len = arrayLike.length; i < len; i++) {
    ……
}
```

上面我们可以看得出来，类数组对象与数组在读取、获取长度、遍历三个方面一样，都能取到，那为什么还叫做类数组对象呢？

因为类数组对象不可以使用数组的方法，比如：

```
arrayLike.push('name');
```

然而上述代码会报错: arrayLike.push is not a function，所以终归还是类数组呐……

arguments对象

arguments比起this来说，要容易理解的多，arguments对象只定义在函数体中，包括了函数的参数和其他属性。

同样地通过上面的例子加以理解，我们直接在函数中去打印出"arguments"这个关键字：

```
function MyFavorite(house, car, money) {

    money = money || '一亿元';

    console.log(arguments);
    console.log("----------------");
}

MyFavorite();
MyFavorite("别墅");
MyFavorite("别墅", "法拉利");
MyFavorite("别墅", "法拉利", "一亿元");
```

打印结果如下：

```
▼Arguments [callee: ƒ, Symbol(Symbol.iterator): ƒ]
  ▶callee: ƒ MyFavorite(house, car, money)
   length: 0
  ▶Symbol(Symbol.iterator): ƒ values()
  ▶__proto__: Object
----------------
▼Arguments ["别墅", callee: ƒ, Symbol(Symbol.iterator): ƒ]
   0: "别墅"
  ▶callee: ƒ MyFavorite(house, car, money)
   length: 1
  ▶Symbol(Symbol.iterator): ƒ values()
  ▶__proto__: Object
----------------
▼Arguments(2) ["别墅", "法拉利", callee: ƒ, Symbol(Symbol.iterator): ƒ]
   0: "别墅"
   1: "法拉利"
  ▶callee: ƒ MyFavorite(house, car, money)
   length: 2
  ▶Symbol(Symbol.iterator): ƒ values()
  ▶__proto__: Object
----------------
▼Arguments(3) ["别墅", "法拉利", "一亿元", callee: ƒ, Symbol(Symbol.iterator): ƒ]
   0: "别墅"
   1: "法拉利"
   2: "一亿元"
  ▶callee: ƒ MyFavorite(house, car, money)
   length: 3
  ▶Symbol(Symbol.iterator): ƒ values()
  ▶__proto__: Object
----------------
```

收藏

symbot ☆ 更多释义»
网络释义
新增元件
相关查询
SYMBOT；symbot

我们可以看到除了类数组的索引属性和length属性之外，还有一个ca
arguments对象的__ *proto* __是指向object的，这也说明了他是个类
组。下面我们进行一一介绍。

return

默认返回值为undefined，用来设定函数的返回值

当一个函数return值之后，return后面的代码将不会被执行；

函数表达式的种类

```
//1.
var a = function(){   console.log(1)  };

//2.
(function(){ console.log('2');})()

//3.
 (function(){console.log( '3' );}())

//4.
+function(){console.log('4');}()

//5.
-function(){console.log('5');}()

//6.
~function(){ console.log('6');}()

//7.
!function(){console.log( '7' )}()
```

获取内部样式

```
getComputedStyle(Obj)
Ojb.currentStyle
```

主流浏览器能运行，IE8及以上也可以

封装

```
            height:200px;
            background-color: deeppink;  /* rgb(255, 20, 147) */
        }
        .wrap1 {
            width:450px;
            height:400px;
            background-color: deeppink;  /* rgb(255, 20, 147) */
        }
        .wrap2 {
            width:60px;
            height:60px;
            background-color: deeppink;  /* rgb(255, 20, 147) */
        }

    </style>
</head>
<body>


    <div class="wrap" id='wrap' style=''></div>
    <div class="wrap1" id='wrap1' style=''></div>
    <div class="wrap2" id='wrap2' style=''></div>



    <script>

        var oWrap = document.getElementById('wrap');
        var oWrap1 = document.getElementById('wrap1');
        var oWrap2 = document.getElementById('wrap2');
        //获取样式;
        // 优化1:
        function getStyle( ele ){
            console.log( ele )
            var cssObj ;
            if( ele.currentStyle ){
                //不是undefined;  ie
                cssObj = oWrap.currentStyle;
            }else{
                //undefied;   主流浏览器【火狐浏览器, 谷歌, QQ,傲游, , , 】
                cssObj = getComputedStyle(ele);
            }
        }
        console.log( cssObj )

        //优化2:
        function getStyle( ele ){
            // console.log( ele )
            var cssObj ;
            //三目:
            // ele.currentStyle ?cssObj = oWrap.currentStyle:cssObj = getComputedStyle(ele);
            return cssObj = ele.currentStyle ? ele.currentStyle: getComputedStyle(ele)

        }

        //优化3:
        function getStyle( ele ){
            // console.log( ele )
            //undefined
            //三目:
            // ele.currentStyle ?cssObj = oWrap.currentStyle:cssObj = getComputedStyle(ele);
            return   ele.currentStyle ||  getComputedStyle(ele)

            // 如果是主流浏览器, ele.currentStyle就是undefined,undefined为假值, 走或取后面的getComputedStyle(ele);
            // 如果为IE浏览器, ele.currentStyle为真, 不会走 || 取后的内容;
        }



        console.log( getStyle( oWrap ).width ,getStyle( oWrap ).height )//200.5px 200px
        console.log( getStyle( oWrap1 ).width ,getStyle( oWrap1 ).height )//450px 400px
        console.log( getStyle( oWrap2 ).width ,getStyle( oWrap2 ).height )//60px 60px
```

构成递归需具备的条件：

1. 子问题须与原始问题为同样的事，且更为简单；
2. 不能无限制地调用本身，须有个出口，化简为非递归状况处理。